Session 7

Advanced Machine Learning Topics 1

Ganjar Alfian
Teknik Elektro dan Informatika
Sekolah Vokasi, Universitas Gadjah Mada
ganjar.alfian@ugm.ac.id

Table of Content

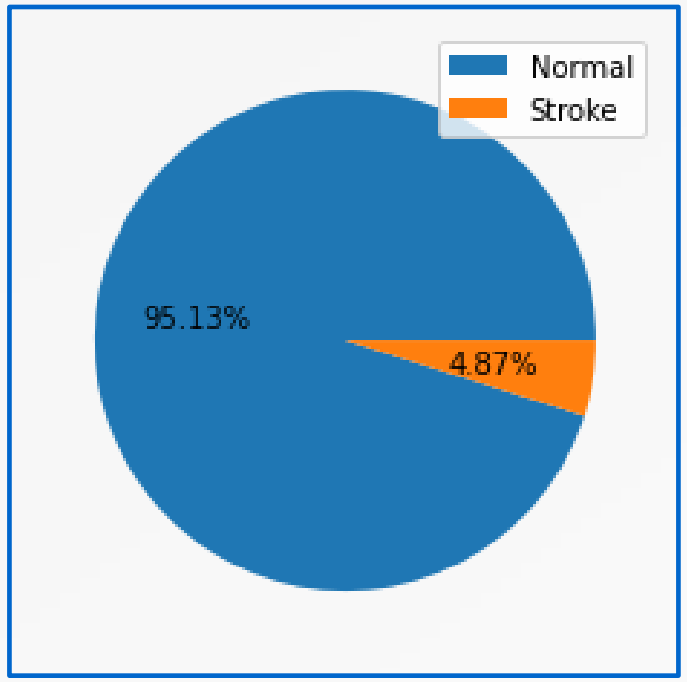
What will We Learn Today?

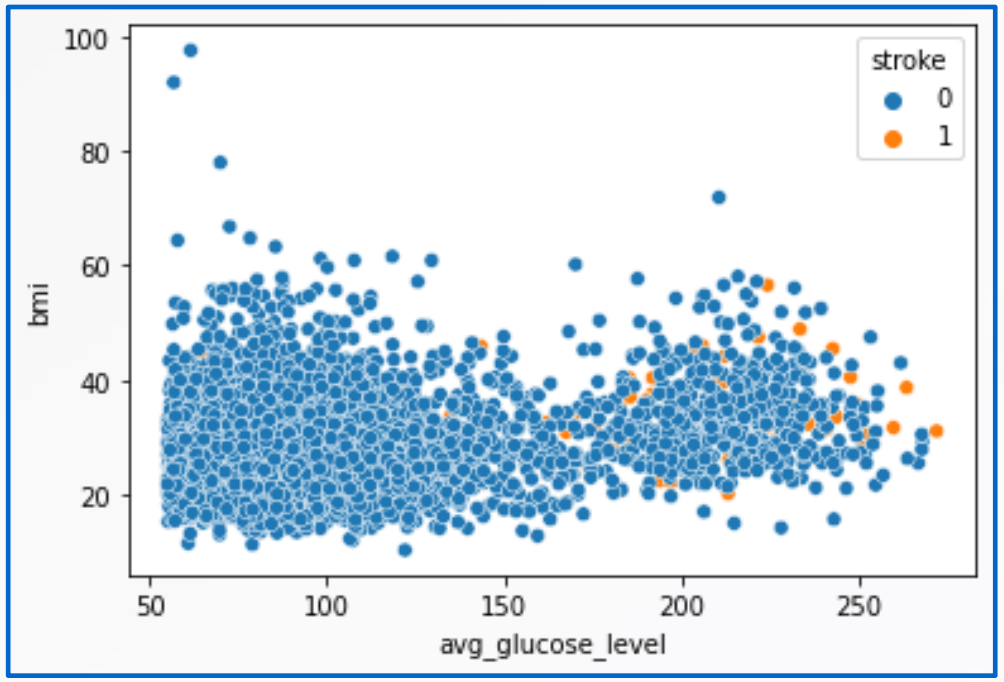
1. Imbalanced Dataset
2. Feature selection (filter, wrapper, embedded)

Imbalanced Dataset

Imbalanced dataset

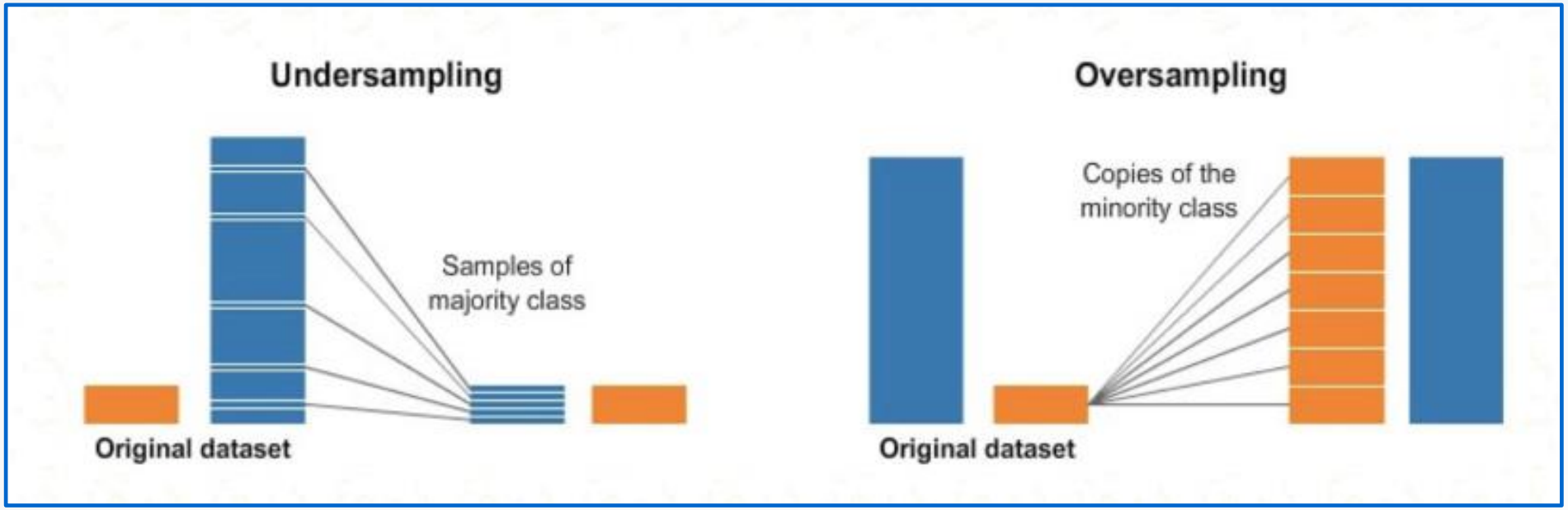
- *Imbalanced dataset* mengacu pada masalah klasifikasi di mana jumlah data per kelas tidak terdistribusi secara merata.
- *https://www.kaggle.com/fedesoriano/stroke-prediction-dataset?select=healthcare-dataset-stroke-data.csv*





How to handle imbalance dataset

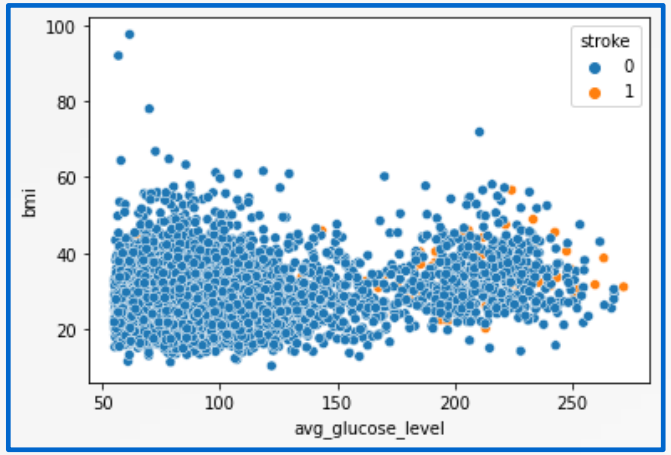
- *Under sampling* = Menyeimbangkan distribusi kelas dengan menghilangkan data dari kelas mayoritas secara acak.
- *Oversampling* = Meningkatkan jumlah instance di kelas minoritas dengan mereplikasinya secara acak.

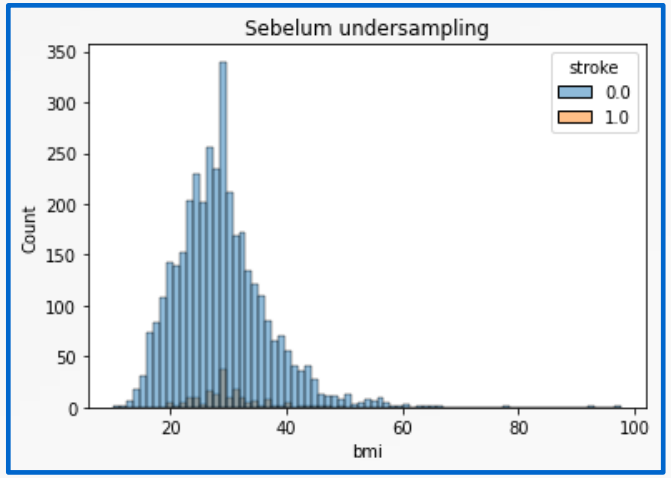


Random Under sampling

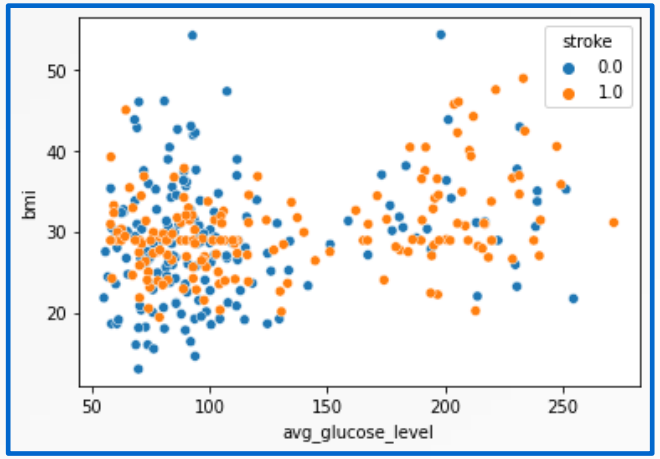
- Membadingkan *training set,* sebelum dan sesudah *undersampling*

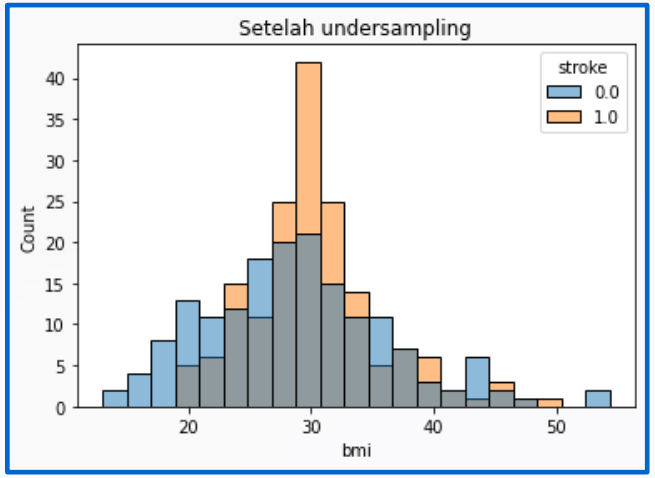
Sebelum





Sesudah



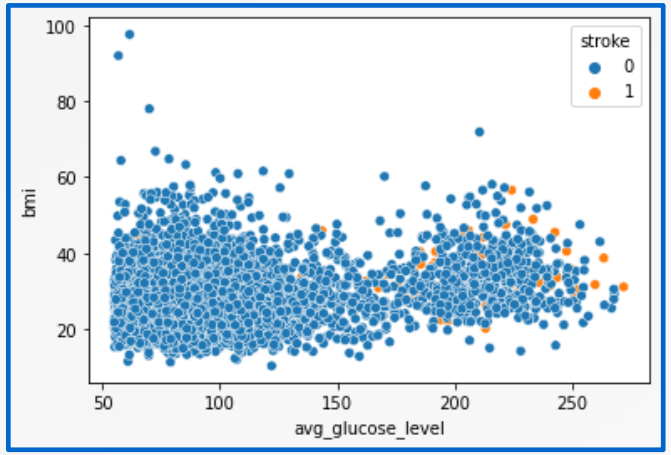


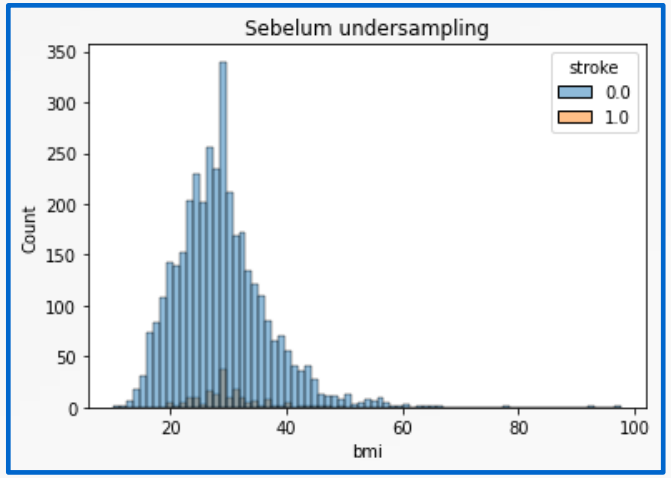

```
Sebelum undersampling
0.0    3663
1.0     169
dtype: int64
Setelah undersampling
1.0    169
0.0    169
dtype: int64
```

Random Over sampling

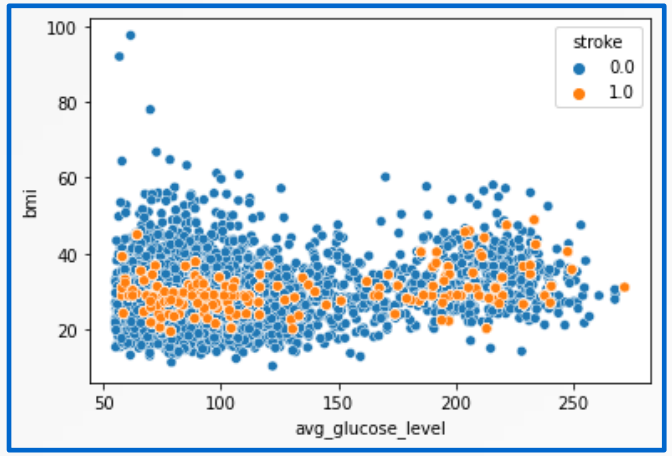
- Membadingkan *training set,* sebelum dan sesudah *oversampling*

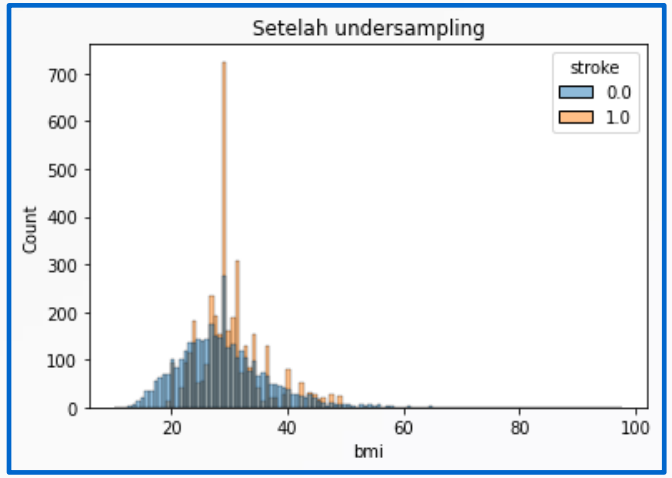
Sebelum





Sesudah

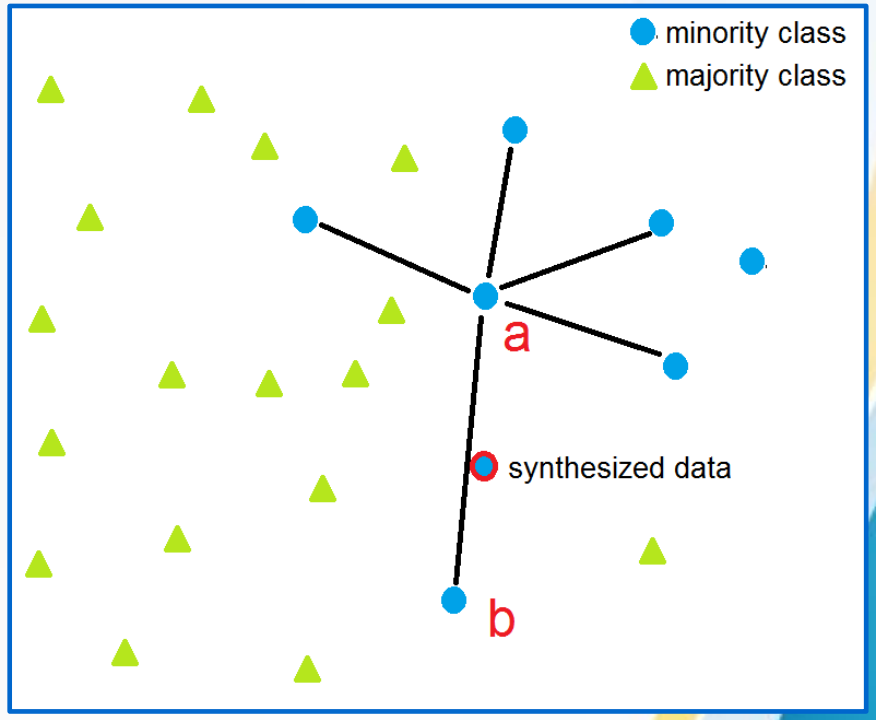





```
Sebelum oversampling
0.0    3663
1.0     169
dtype: int64
Setelah oversampling
1.0    3663
0.0    3663
dtype: int64
```

SMOTE

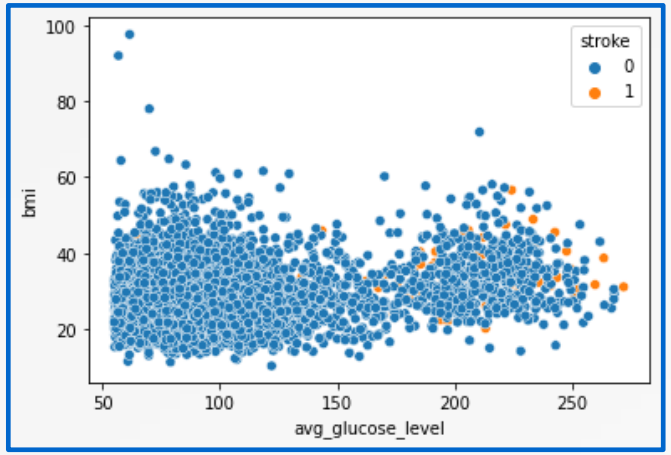
- *SMOTE = Synthetic Minority Oversampling Technique*
- Pendekatan *oversampling* yang menciptakan sampel kelas minoritas secara sintetis.
- Cara kerja:
  - Contoh acak dari kelas minoritas **a** dipilih terlebih dahulu.
  - Kemudian k dari tetangga terdekat (nearest neighbour) untuk contoh tersebut ditemukan (biasanya k=5).
  - Tetangga **b** yang dipilih secara acak.
  - Data sintetik **c** dibuat pada titik yang dipilih secara acak diantara dua data tersebut (**a** dan **b**)

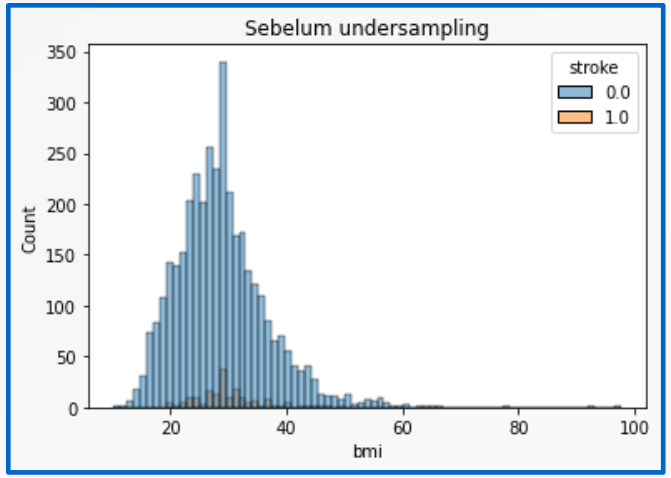


SMOTE

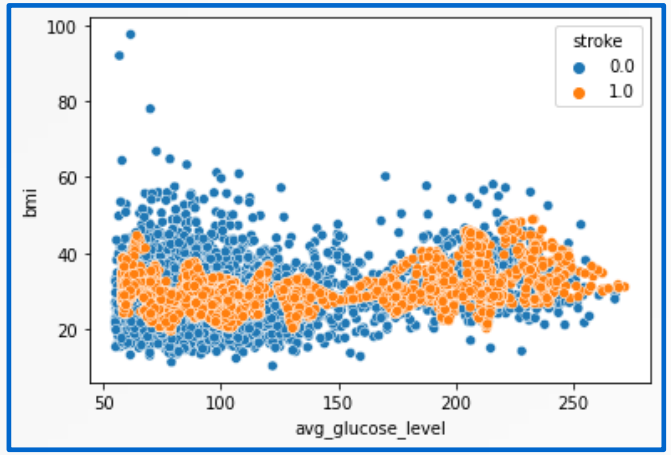
- Membadingkan *training set,* sebelum dan sesudah *SMOTE*

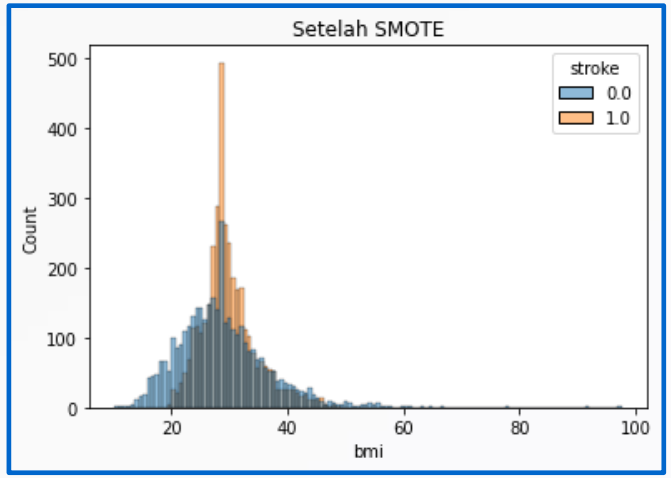
Sebelum





Sesudah

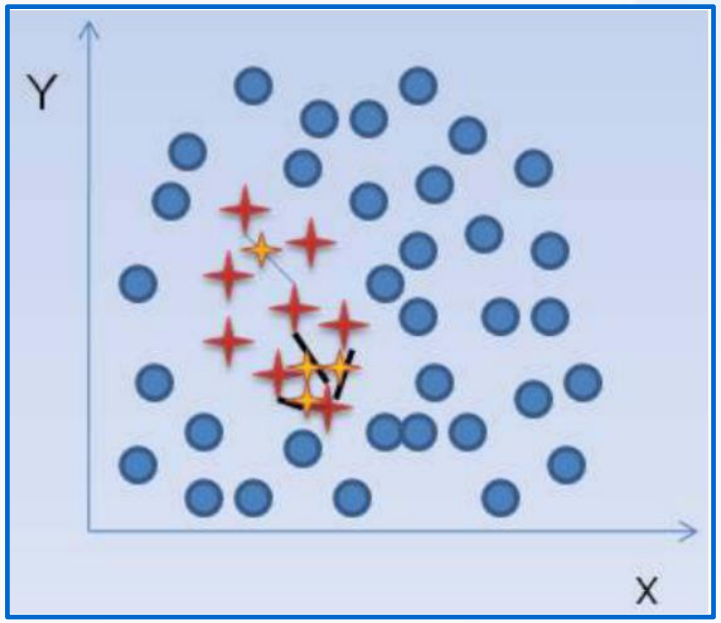


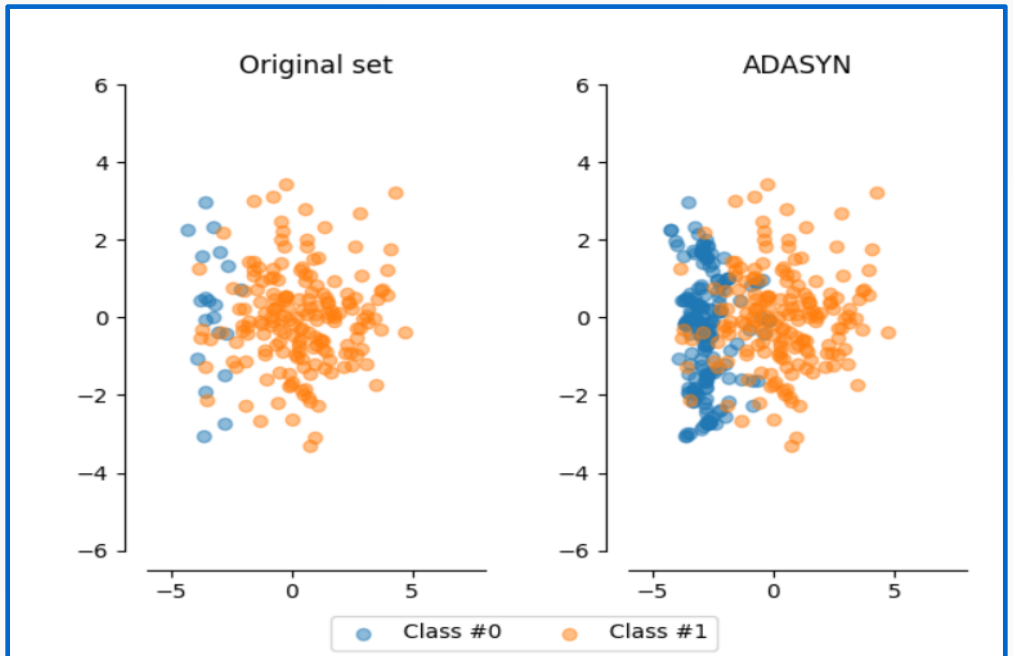



```
Sebelum SMOTE
0.0    3663
1.0     169
dtype: int64
Setelah SMOTE
1.0    3663
0.0    3663
dtype: int64
```

ADASYN

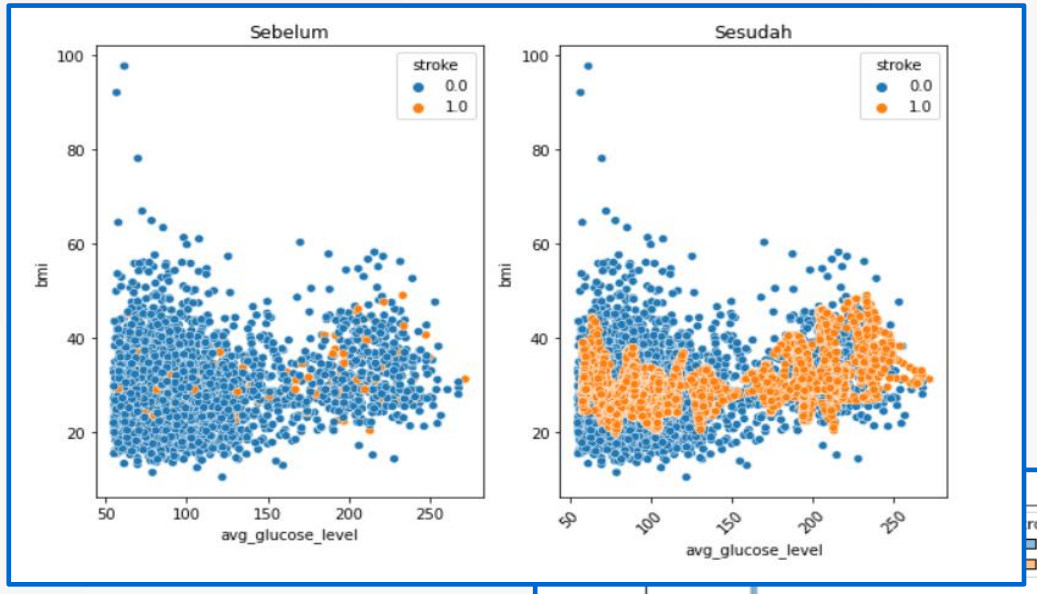
- *ADASYN = Adaptive Synthetic Sampling Approach for Imbalanced Learning*
- Ide penting dari ADASYN adalah menggunakan pembobotan untuk contoh kelas minoritas
- ADASYN akan fokus pada sampel yang sulit untuk diklasifikasikan sementara SMOTE tidak akan membuat perbedaan apa pun.

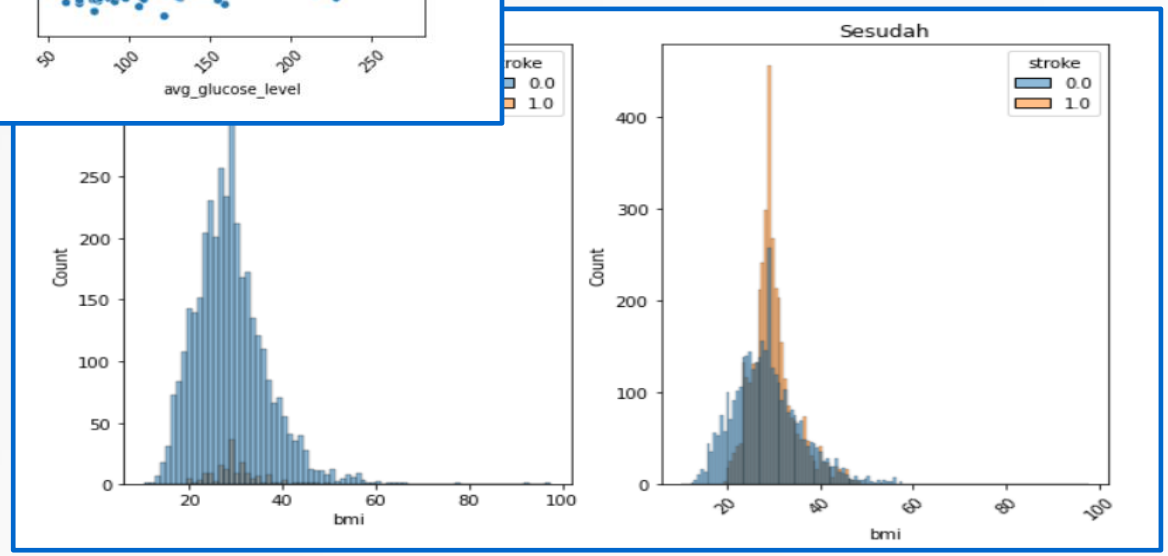




ADASYN

- Membadingkan *training set,* sebelum dan sesudah *ADASYN*



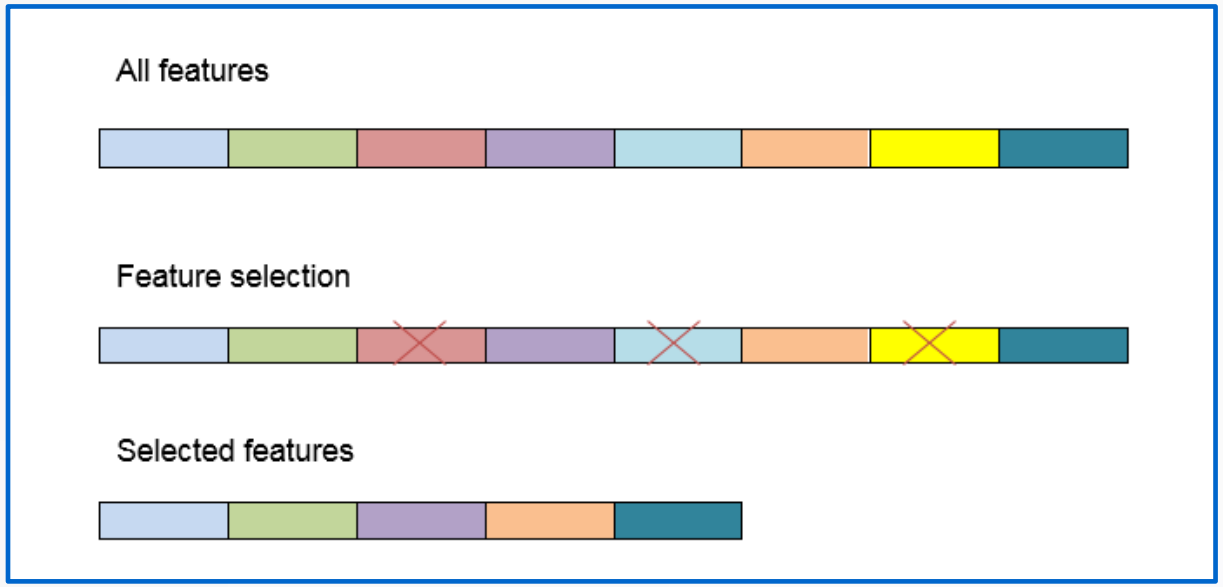



```
Sebelum ADASYN
0.0    3663
1.0     169
dtype: int64
Setelah ADASYIN
1.0    3676
0.0    3663
dtype: int64
```

Feature Selection

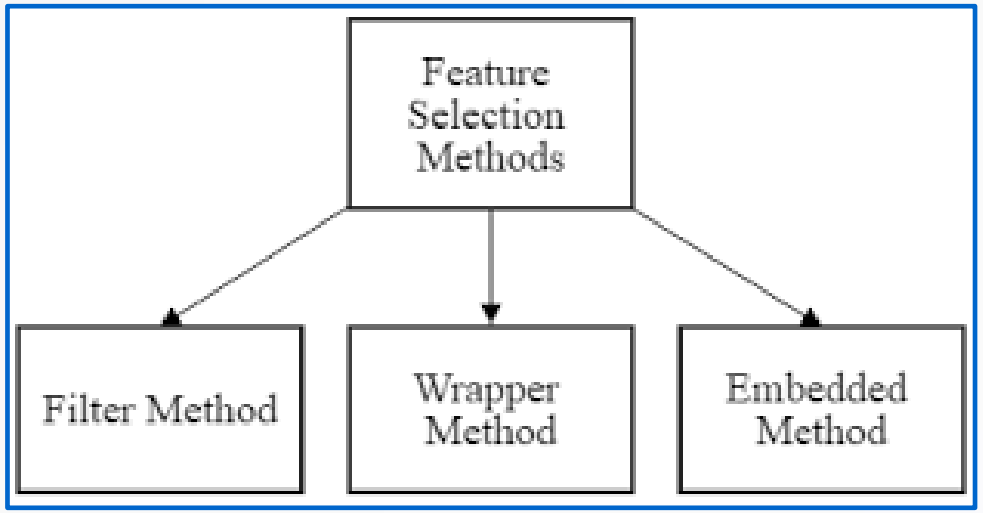
Feature Selection

- Feature selection is a process of selection a subset of Relevant Features (Variables or Predictors) from all features, which is used to make Model Building.
- Feature Selection :
  - Removing irrelevant data
  - Can reduce computation time,
  - Improve learning accuracy
  - Better understanding for the learning model or data.



Feature Selection

- Filter methods
  - measure the relevance of features by their correlation with the target variable.
- Wrapper methods
  - utilize a learning machine to measure the usefulness of a subset of features according to their predictive power.
- Embedded methods
  - perform feature selection in the process of training based on specific learning machines.



Reading CSV dataset

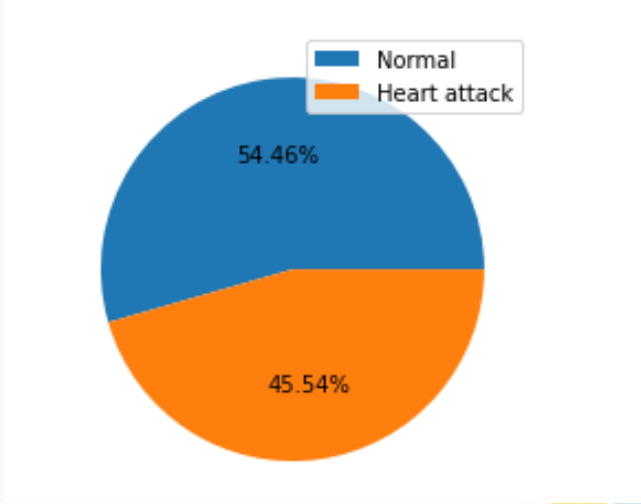
- First, download the dataset
  - https://www.kaggle.com/rashikrahmanpritom/heart-attack-analysis-prediction-dataset?select=heart.csv
- Upload to google drive
- Read the CSV file from your google drive.

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/
```

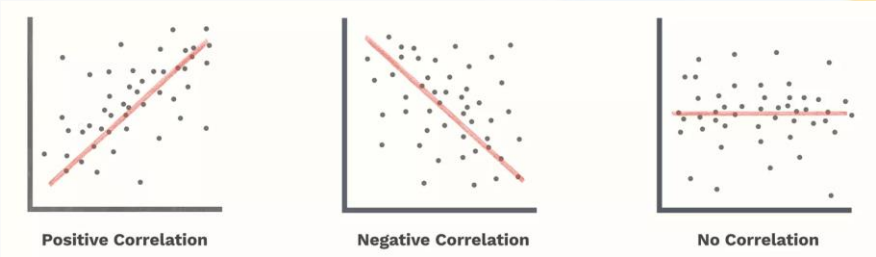
```
import pandas as pd
df = pd.read_csv('/content/gdrive/MyDrive/Data Science Digital Skola/heart.csv', delimiter = ",")
df.head()
#df.columns
```

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1



Filter method

- By correlating the feature with the target (what we are predicting).
- The features with the highest correlation are the best.
- We find out the correlation between target variable with respect to independent variables. (Y→ X1) ,(Y →X2),(Y →X3),…(Y →Xn)



- Analysis of Variance or ANOVA.
- ANOVA is a statistical method that analyses variances to determine if the means from more than two populations are the same.

Hypothesis

$$H_0 : \bar{x}_1 = \bar{x}_2 = \bar{x}_3 = \ldots = \bar{x}_k$$
$$H_A : \text{At least one of the groups means differ}$$

The test statistic is the F-statistic and compares the mean square between samples (MS_B) to the mean square within sample (MS_W). This *F-statistic* can be calculated using the following formula:

$$F = \frac{MS_B}{MS_W}$$

Source : https://www.pythonfordatascience.org/anova-python/

Filter method using ANOVA

```
from sklearn.feature_selection import SelectKBest, f_classif

df_X = df.drop(['output'],axis=1)
df_y = df['output']

X = df_X.astype(float).values
y = df_y.astype(float).values

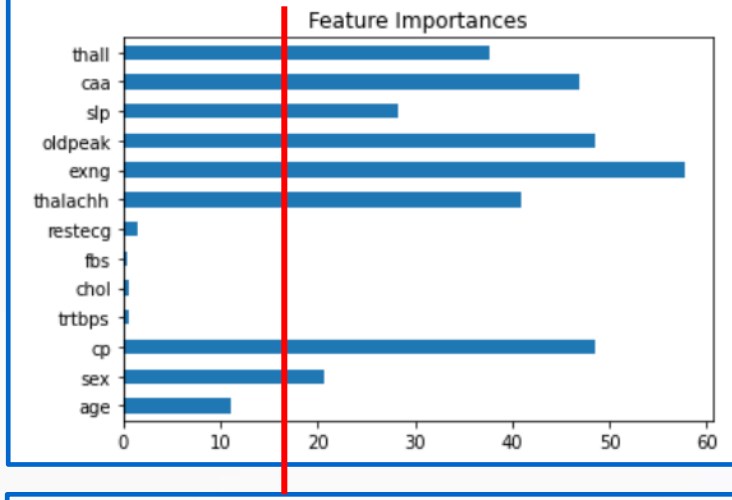
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

filter = SelectKBest(f_classif, k=8)
filter.fit(X_train, y_train)
X_train_new = filter.transform(X_train)
X_test_new = filter.transform(X_test)

print('Sebelum feature selection ', X.shape)
print('Setelah feature selection ', X_train_new.shape)
print('Score of features ', filter.scores_)
print('Score of features ', filter.pvalues_)

feat_importances = pd.Series(filter.scores_, index=df_X.columns)
feat_importances.plot(kind='barh')
plt.title('Feature Importances')
plt.show()
```

```
Sebelum feature selection  (303, 13)
Setelah feature selection  (212, 8)
Score of features  [11.10851109 20.58085872 48.59574373  0.59397382  0.59877117  0.31089985
  1.46204906 40.9269554  57.695455   48.50771938 28.24118453 46.81205511
 37.68642199]
Score of features  [1.01552795e-03 9.60995300e-06 4.00554531e-11 4.41752777e-01
 4.39917491e-01 5.77722396e-01 2.27963944e-01 1.01086744e-09
 9.93392466e-13 4.15424082e-11 2.72611454e-07 8.40617281e-11
 4.08990656e-09]
```



Selected features = 8

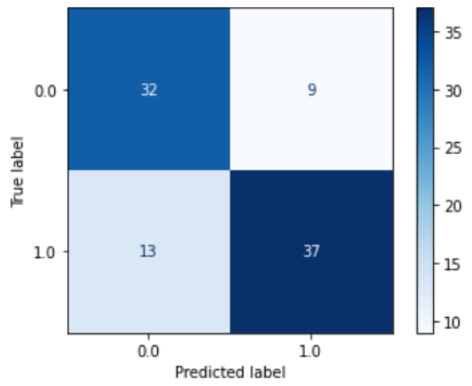
Selected features = sex, cp, thalachh, exng, oldpeak, slp, caa, thall

DT + ANOVA

```
model=DecisionTreeClassifier(random_state=42)
model.fit(X_train_new, y_train)
y_pred = model.predict(X_test_new)

print('Accuracy ',metrics.accuracy_score(y_test, y_pred))
print('Precision ',metrics.precision_score(y_test, y_pred, average='macro'))
print('Recall ',metrics.recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', metrics.confusion_matrix(y_test, y_pred))
plot_confusion_matrix(model, X_test_new, y_test, cmap=plt.cm.Blues)
plt.show()
```

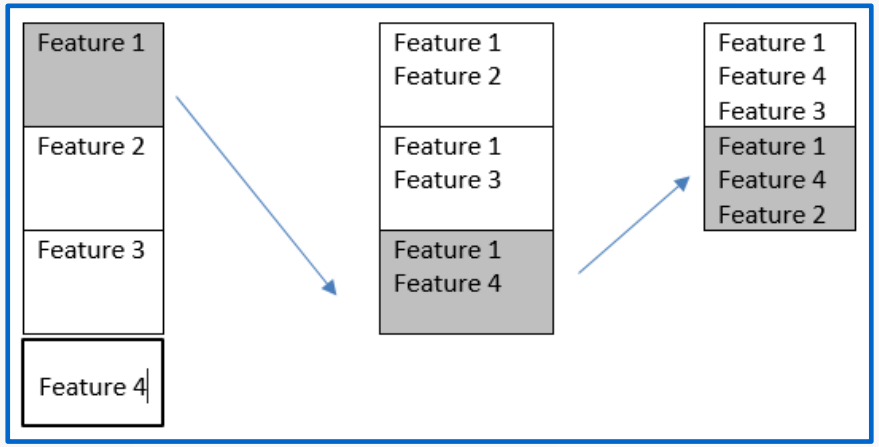
```
Accuracy  0.7582417582417582
Precision  0.7577294685990339
Recall  0.7602439024390244
Confusion matrix  [[32  9]
 [13 37]]
```



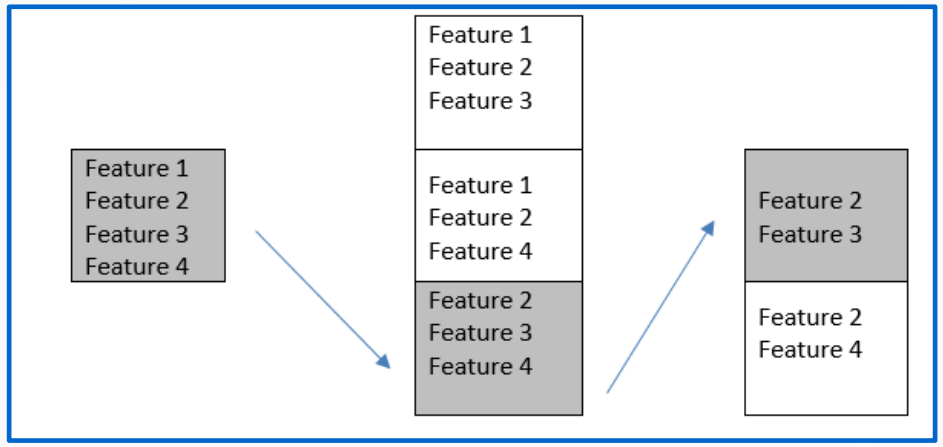
Classifier	Feature selection method	Accuracy
DT	-	0.736
DT	ANOVA	0.758

Wrapper method

- The wrapper method will find the best combination of variables.
- They are:
  - Subset Selection,
  - Forward Stepwise, and
  - Backward Stepwise or Recursive Feature Elimination (RFE).
- It tests each feature against test models that it builds with them to evaluate the results.
- Very computationally intensive.
- It is not recommended that this method be used on a high number of features.



Forward stepwise



Backward stepwise or RFE

Wrapper method using RFE

```
from sklearn.feature_selection import RFE
from sklearn.svm import SVC

df_X = df.drop(['output'],axis=1)
df_y = df['output']

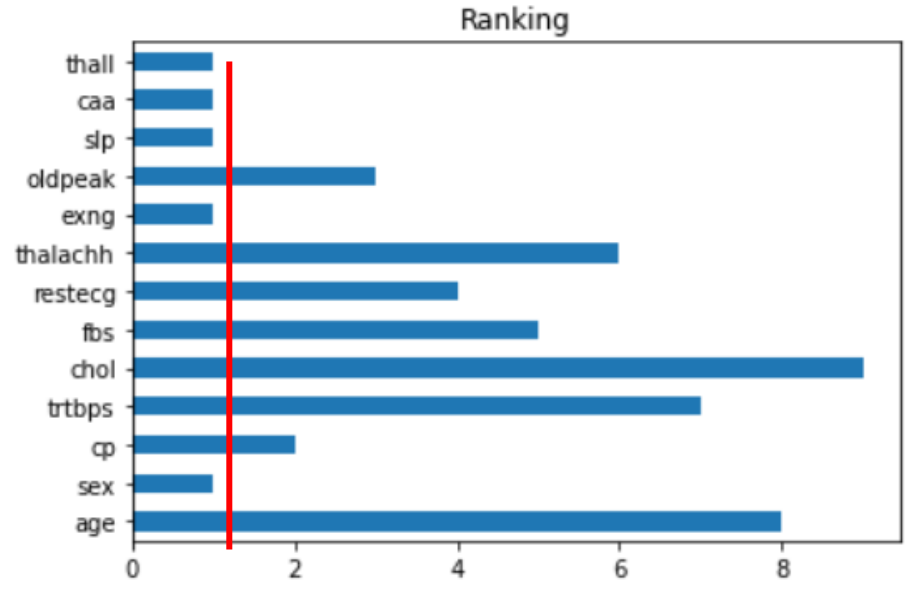
X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

estimator = SVC(kernel="linear")
wrapper = RFE(estimator, n_features_to_select=5)
wrapper.fit(X_train, y_train)
X_train_new = wrapper.transform(X_train)
X_test_new = wrapper.transform(X_test)

print('Sebelum feature selection ', X.shape)
print('Setelah feature selection ', X_train_new.shape)
print('Score of features ', wrapper.ranking_)

feat_importances = pd.Series(wrapper.ranking_, index=df_X.columns)
feat_importances.plot(kind='barh')
plt.title('Ranking')
plt.show()
```

```
Sebelum feature selection  (303, 13)
Setelah feature selection  (212, 5)
Score of features  [8 1 2 7 9 5 4 6 1 3 1 1 1]
```



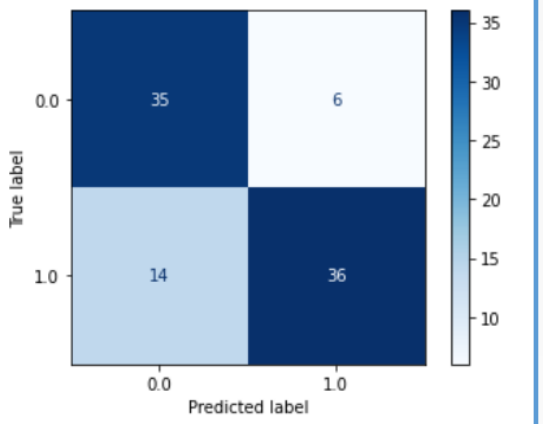
Selected features = 5
Selected features = sex, exng, slp, caa, thall

DT + RFE

```
model=DecisionTreeClassifier(random_state=42)
model.fit(X_train_new, y_train)
y_pred = model.predict(X_test_new)

print('Accuracy ',metrics.accuracy_score(y_test, y_pred))
print('Precision ',metrics.precision_score(y_test, y_pred, average='macro'))
print('Recall ',metrics.recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', metrics.confusion_matrix(y_test, y_pred))
plot_confusion_matrix(model, X_test_new, y_test, cmap=plt.cm.Blues)
plt.show()
```

```
Accuracy  0.7802197802197802
Precision  0.7857142857142857
Recall  0.786829268292683
Confusion matrix  [[35  6]
 [14 36]]
```

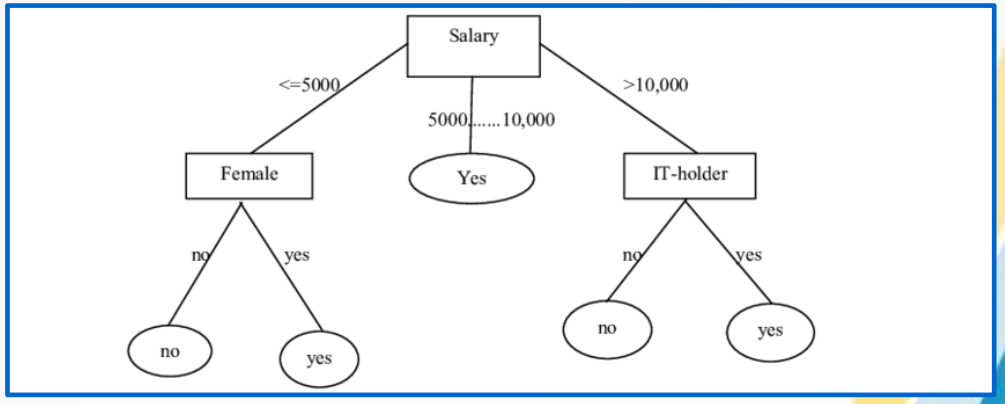


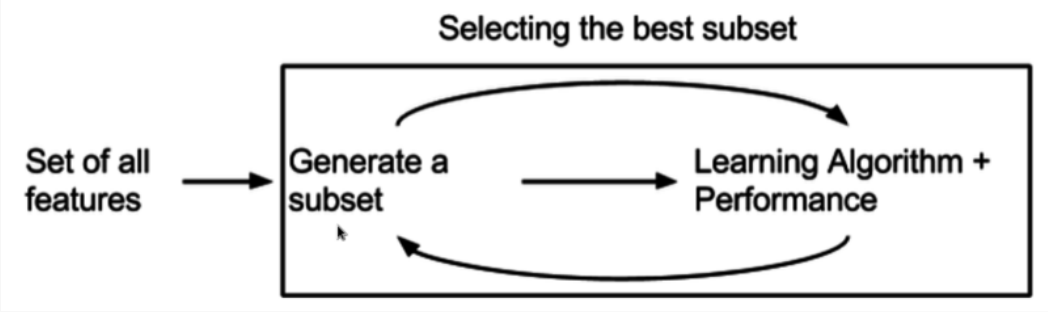
Classifier	Feature selection method	Accuracy
DT	-	0.736
DT	ANOVA	0.758
DT	RFE	0.780

Embedded method

- The feature selection algorithm is integrated as part of the learning algorithm.
- The most Common embedded technique are the tree algorithm's like RF, ExtraTree and so on.
- The algorithm tries all possible ways of split for all the features and chooses the one that splits the data best.

- Advantages
  - The interaction of features like wrapper methods do.
  - They are faster like filter methods.
  - They are more accurate than filter methods.
  - They find the feature subset for the algorithm being trained.





Embedded method using ExtraTree

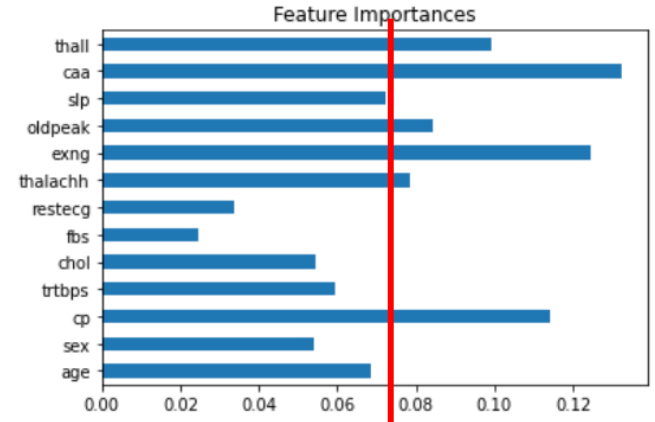
- First, download the dataset : https://www.kaggle.com/rashikrahmanpritom/heart-attack-analysis-prediction-dataset?select=heart.csv

```
X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
clf = ExtraTreesClassifier(random_state=2)
clf = clf.fit(X_train, y_train)
embed = SelectFromModel(clf, prefit=False)
embed.fit(X_train, y_train)
X_train_new = embed.transform(X_train)
X_test_new = embed.transform(X_test)

print('Sebelum feature selection ', X.shape)
print('Setelah feature selection ', X_train_new.shape)
print('Score of features ', clf.feature_importances_)
print(np.mean(clf.feature_importances_))
feat_importances = pd.Series(clf.feature_importances_, index=df_X.columns)
feat_importances.plot(kind='barh')
plt.title('Feature Importances')
plt.show()
```

```
Sebelum feature selection  (303, 13)
Setelah feature selection  (212, 6)
Score of features  [0.06864698 0.05419113 0.1142734  0.05924801 0.05436202 0.02454468
 0.03361371 0.07863889 0.12434475 0.08415603 0.07227328 0.13229248
 0.09941463]
0.07692307692307693
```



Selected features = 6
Selected features = cp, thalachh, exng, oldpeak, caa, thall

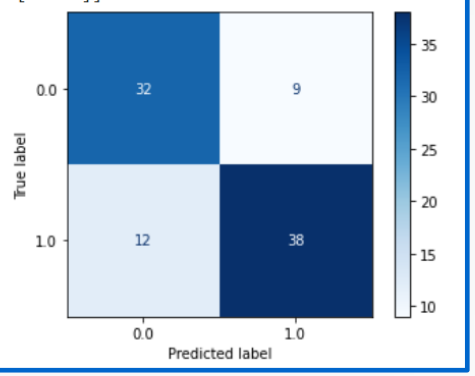
DT + ExtraTree

- First, download the dataset : https://www.kaggle.com/rashikrahmanpritom/heart-attack-analysis-prediction-dataset?select=heart.csv

```
model=DecisionTreeClassifier(random_state=42)
model.fit(X_train_new, y_train)
y_pred = model.predict(X_test_new)

print('Accuracy ',metrics.accuracy_score(y_test, y_pred))
print('Precision ',metrics.precision_score(y_test, y_pred, average='macro'))
print('Recall ',metrics.recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', metrics.confusion_matrix(y_test, y_pred))
plot_confusion_matrix(model, X_test_new, y_test, cmap=plt.cm.Blues)
plt.show()
```

```
Accuracy  0.7692307692307693
Precision  0.7678916827852997
Recall  0.7702439024390244
Confusion matrix  [[32  9]
 [12 38]]
```

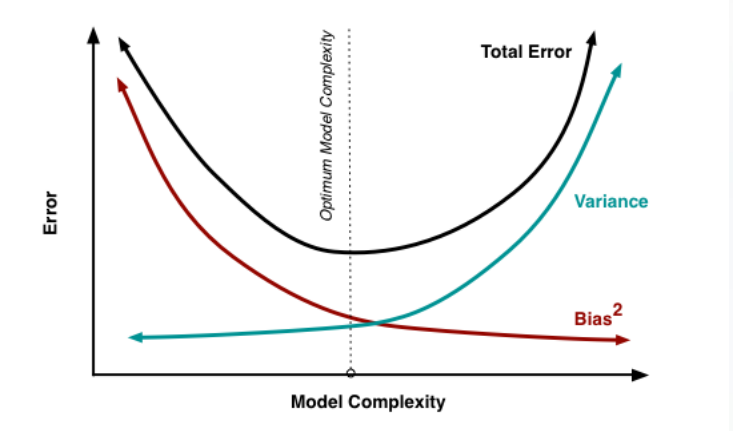


Classifier	Feature selection method	Accuracy
DT	-	0.736
DT	ANOVA	0.758
DT	RFE	0.780
DT	ExtraTree	0.769

Comparison

- In the following table, let us explore the comparison of these three methods of feature selection.

	Filter method	Wrapper method	Embedded method
What is it?	Uses proxy measure	Uses predictive model	Feature selection is embedded in the model building phase
Speed	Computationally faster	Slower	Medium
Overfitting	Avoids overfitting	Prone to overfitting	Less prone to overfitting
Performance	Sometimes may fail to select best features	Better performance	Good performance



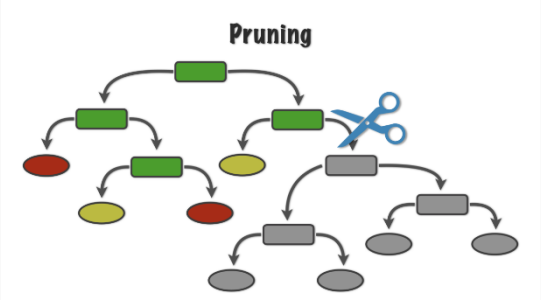
http://www.datasciencesmachinelearning.com/2019/10/feature-selection-filter-method-wrapper.html

Next step

- Need to optimize number of features

Classifier	Feature selection method	Number of Features	Accuracy
DT	-	13	0.736
DT	Filter (ANOVA)	8	0.758
DT	Wrapper (RFE)	5	0.780
DT	Embedding (ExtraTree)	6	0.769

- Need to avoid overfitting for the classifier
  - DT + ExtraTree
    - Accuracy of training = 0.99
    - Accuracy of testing = 0.769
  - DT (after pruning) + ExtraTree
    - Accuracy of training = 0.93
    - Accuracy of testing = 0.82




```
model=DecisionTreeClassifier(random_state=42, max_depth=6)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Lets Practice!

Thank You